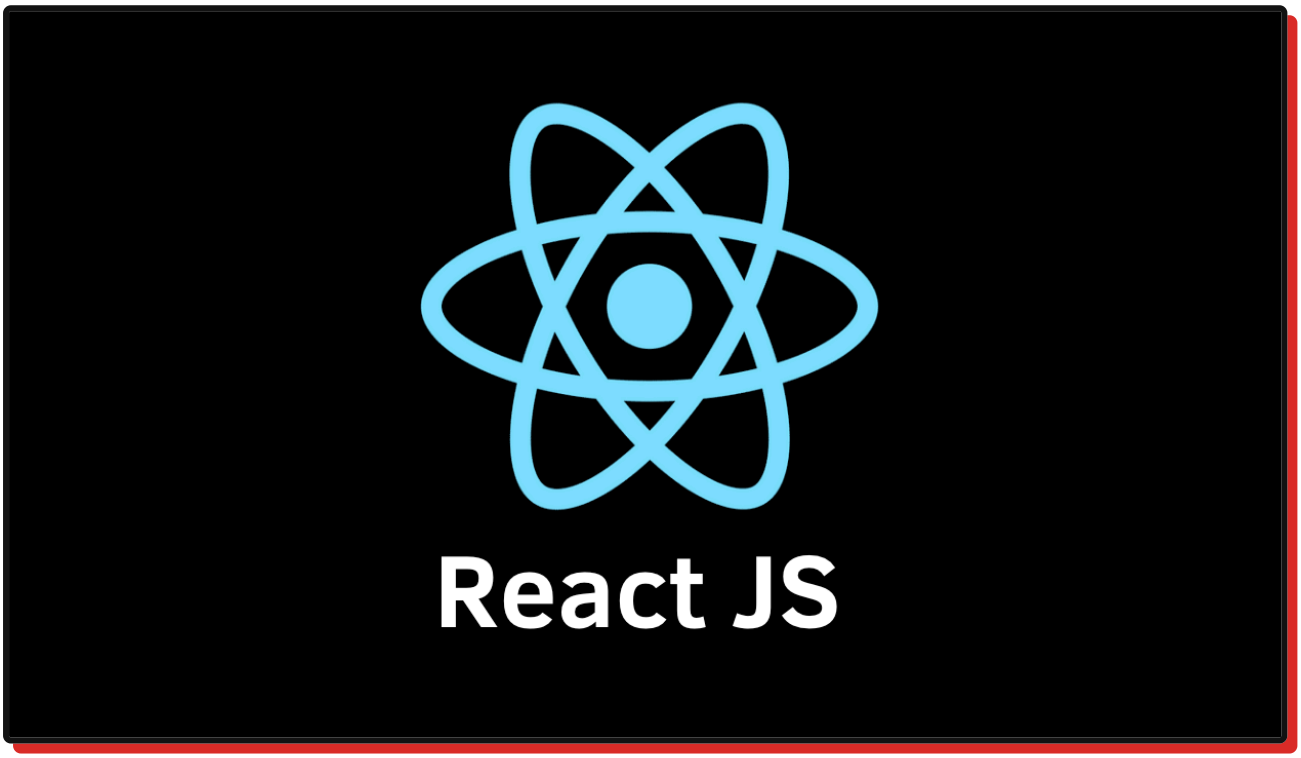
PROYECTO 3 EN RAYA: EXPLICACIÓN

Sergio Lozano Molina
2ºDAM



RESUMEN

En esta práctica lo que llevaremos a cabo será trabajar con **React** para desarrollar el juego del **Tres en raya**, una aplicación construida a partir del tutorial oficial de React. En este documento indicaremos donde se ha guardado el desarrollo del proyecto en GitHub y responderemos a las preguntas teóricas sobre los conceptos principales del framework (`componentes`, `JSX`, `props`, `useState`, `elevación del estado e inmutabilidad`).

ÍNDICE

Tarea 1 Adjuntar el código en repositorio Github

Tarea 2 Responder a las siguientes preguntas

- Pregunta 1 Componentes
- Pregunta 2 JSX
- Pregunta 3 Props
- Pregunta 4 Estado con useState
- Pregunta 5 Manejo de eventos
- Pregunta 6 Elevar el estado (Lifting State Up)
- Pregunta 7 Inmutabilidad
- Pregunta 8 Renderizado condicional
- Pregunta 9 Funciones flecha como callbacks

TAREA 1: ADJUNTAR EL CÓDIGO EN REPOSITORIO GITHUB

El código del proyecto se encuentra en el siguiente enlace que lleva al repositorio de Github:
https://github.com/1oza16/Evaluacion-continua-react.git

TAREA 2: RESPONDER A LAS SIGUIENTES PREGUNTAS

PREGUNTA 1: COMPONENTES

En esta pregunta analizaremos que es un un **componente** en **React** y la relación entre los componentes `Square`, `Board` y `Game`.

Un componente en react consiste en un pieza de código la cual es reutilizable e independiente y que permite encapsular su propia estructura, lógica y estilo para devolver un elemento de interfaz (Conocido como JSX).

- `Square`: Este es el componente hijo más pequeño y que en este caso, representa una única casilla del tablero y se encarga de renderizar el botón con su valor (`'X'`, `'O'` o vacío). Además es capaz de responder al clic del usuario para cambiar dicho valor.

- `Board`: Este es el componente intermedio el cual compone y organiza 9 componentes `Square` haciendo una cuadrícula de tres en raya. También administra las reglas de la partida (validar victorias, saber a quién le toca jugar) y pasa a cada casilla los datos que necesita.

- `Game`: Este el componente contenedor, por lo tanto la raíz. Se encraga de mantener el historial de movimientos y renderiza tanto el `Board` como la lista para viajar en el tiempo entre jugadas.

PREGUNTA 2: JSX

JSX consiste en una extensión de sintaxis la cual nos permite escribir HTML directamente en el código JavaScript. Esto permite definir la apariencia de la interfaz de forma más intuitiva.

Una de las reglas y que es imprescindible a cumplir es que todo componente se que **debe devolver un solo elemento raíz**. Por lo que si se quiere devolver varios elementos seguidos, hay que envolverlos dentro de un contenedor, ya sea dentro de un `<div>` o mediante `<> ... </>`.
Un ejemplo de ello es:

```
return (
  <div className="board-row">
    <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
    <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
    <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
  </div>
);
```

PREGUNTA 3: PROPS

Las **props** en **React** consisten en los parámetros de entrada que un componente padre le pasa a su componente hijo, con el objetivo de transmitirle información o funciones callback. Algunas de sus cualidades son:

- **Solo lectura:** Son inmutables dentro del componente hijo, por lo que el hijo no debe modificar las props que recibe, garantizando que el flujo de datos sea mas fácil de gestionar y trabajar.
- **Uso en `Square`:** `Square` recibe dos props desde `Board`: `value` (el texto `'X'`, `'O'` o `null` que debe mostrar la casilla) y `onSquareClick` (la función que debe ejecutarse cuando el usuario pulsa esa casilla).

PREGUNTA 4: ESTADO CON USESTATE

En esta pregunta veremos para qué sirve el hook `useState` y cuáles son los dos elementos que devuelve al utilizarlo en **React**.

El hook `useState` nos permite añadir un estado local a un componente para que pueda guardar información en memoria. De esta forma, cada vez que esa información cambia, la pantalla se actualiza de forma automática con los nuevos datos.

- **Valor del estado:** Es el primer elemento devuelto y representa el dato guardado en ese instante (Por ejemplo, el turno actual o el historial de tableros).

- **Función actualizadora:** Es el segundo elemento devuelto y consiste en la función que nos permite modificar dicho valor, indicándole a React que debe volver a renderizar el componente.

PREGUNTA 5: MANEJO DE EVENTOS

En este apartado explicaremos cómo se asocia una función a un evento en **JSX**, tomando como referencia el botón del componente `Square`.

Para vincular una función a un evento en JSX (como un clic de ratón), se pasa dicha función como una propiedad utilizando la sintaxis ***camelCase*** (como `onClick`) dentro del elemento con el que interactúa el usuario.

Un ejemplo que podemos observar **con** `handleClick` **en** `Square` es como `Square`, se le pasa la prop `onSquareClick` directamente al atributo `onClick` del elemento `<button>`.

De esta forma cuando el usuario hace clic en el botón de la casilla, este dispara la función `onSquareClick`, que a su vez ejecuta el manejador `handleClick(i)` definido en el componente padre (`Board`).
El ejemplo en cuestion es:

```
<button className="square" onClick={onSquareClick}>
```

PREGUNTA 6: ELEVAR EL ESTADO (LIFTING STATE UP)

Aquí analizaremos en qué consiste la técnica de **elevar el estado** y por qué fue necesaria en el tutorial del tres en raya.

Elevar el estado consiste en extraer la declaración del estado de los componentes hijos y moverla hacia el componente padre común más cercano.

- **Mover el estado de `Square` a `Board`:** Esto fue necesario ya que cada casilla por sí sola solo conoce su propio estado aislado. Por lo que para poder validar las combinaciones ganadoras o controlar a qué jugador le toca mover en cada turno, el componente `Board` necesita tener una visión global de las 9 casillas al mismo tiempo.

PREGUNTA 7: INMUTABILIDAD

En esta pregunta explicaremos qué significa tratar los datos de forma **inmutable** en **React** y por qué se realiza una copia del array `squares`.

Tratar los datos de forma inmutable significa que no debemos modificar directamente los objetos o arrays originales que tenemos en memoria, sino crear una copia con los cambios aplicados cada vez que queramos actualizarlos.

- **Conservar el historial:** Al no alterar los arrays originales de las jugadas con métodos como `slice()`, podemos mantener intactas las versiones anteriores de la partida, lo que facilita implementar la función de volver atrás a movimientos pasados.
- **Detección rápida de cambios:** Permite a React comparar referencias en memoria de forma muy eficiente, detectando al instante si un objeto ha cambiado para volver a renderizar la pantalla de manera optimizada.

PREGUNTA 8: RENDERIZADO CONDICIONAL

Ahora explicaremos cómo se muestra un mensaje distinto en pantalla según la situación de la partida mediante el **renderizado condicional**.

El renderizado condicional consiste en emplear lógica de JavaScript para decidir qué interfaz o contenido debe mostrarse al usuario en función del estado actual de la aplicación.

- **Uso de `if / else` en `status`:** Dentro del componente `Board`, se evalúa la función `calculateWinner(squares)`. Si devuelve un ganador, la estructura `if` asigna el texto del vencedor a la variable `status`; si devuelve `null`, pasa al `else` para construir el texto indicando a quién le toca mover en base a la variable `xIsNext`.

PREGUNTA 9: FUNCIONES FLECHA COMO CALLBACKS

Por último, explicaremos por qué se utiliza `() => handleClick(0)` al pasar la función al evento `onClick` de un `Square` en lugar de llamarla directamente.

Utilizar una función flecha como `() => handleClick(0)` nos permite pasar una función que **solo se ejecutará cuando el usuario haga clic** en la casilla.

- **Problema de la llamada directa:** Si escribiéramos `onClick={handleClick(0)}`, la función se ejecutaría inmediatamente en el momento en que React dibuja el componente en pantalla. Al ejecutarse, actualizaría el estado del juego al instante, lo que provocaría un nuevo renderizado y volvería a ejecutar la función, desencadenando un bucle infinito que bloquea la aplicación.